

- Curric Spring Academy - BROADCOM
- Spring e Springboot não
ambos frameworks de java!

• Spring: Não oferece uma caixa
de ferramentas com muitos
ferramentas para construir qualquer
coisa! Exemplo:

Spring MVC, Spring Data for databases,
Spring Security para authentication
e authorization. Enfim não oferece uma
suite de ferramentas para nos auxiliar
na construção de uma aplicação! Porém
esta versatilidade vem com um custo!

Uma aplicação Spring requer muita
configuração que precisa ser feita de forma
manual configurando cada componente para rodar!

- Spring boot: Faz o trabalho com Spring no muito simples! Ele já vem com muitas configurações e dependências já pre-configuradas!

O que torna o trabalho muito mais rápido para as iniciativas! Spring Boot vem também com um servidor web integrado, permitindo criar e implementar aplicações web facilmente sem a necessidade de um servidor externo!

- Perceba:
Spring é poderoso, com bom desempenho e muita flexibilidade! Porém é complicado para configurar! Spring Boot é mais otimizado e uma versão simplificada do Spring para nos ajudar a iniciar mais rápido!

• Iniciando Spring Boot:

- Quando iniciamos um Spring Boot o primeiro passo recomendado é:
"Initializr"

Poderemos pensar no Spring Initializer como um carrinho de compras para escolhermos quais dependências a nossa aplicação precisa! Ele irá gerar um Spring Boot já pronto para iniciar de maneira rápida e fácil!

• API Contracts e JSON:

- Quando vamos construir um API muitos pontos devem ser respondidos:
 - Quem deverá consumir a API?
 - Quais dados o consumidor precisa enviar?
 - Que tipo de dado a API deve retornar?

• Estes cenários e promessas e o compromisso
devem ser reunidos para discutir e chegar
em algum acordo! Eles também devem ser
documentados o que é decidido, não apenas
para ter o documento, mas para que também
de forma que permita a geração (ou regeneração)
de texto automatizado com base em regras
decisões! É aqui que entra o conceito de contratos!

- Contratos de API:

Existem vários tipos de contratos bem como
entre um provedor e um consumidor
que consomem abstratamente como
interagir com o centro,

Exemplos:

Consumer Driven Contracts
and

Provider Driven Contracts

Não iremos nos aprofundar neste agora, usaremos um tipo de contrato mais simples o API contracts,

Definidos os acordos entre prometer e consumidor para a API que iremos criar neste curso teremos então o contrato feito para "Family CashCard"

• Nesse simples contrato:

Request

URI: /cashcards/{id}

HTTP Verb: GET

Body: None

Response:

HTTP Status:

200 OK if the user is authorized and the Cash Card was successfully retrieved

401 UNAUTHORIZED if the user is unauthenticated or unauthorized

404 NOT FOUND if the user is authenticated and authorized but the Cash Card cannot be found

Response Body Type: JSON

Example Response Body:

```
{  
  "id": 99,  
  "amount": 123.45  
}
```

• O que é JSON?

JSON = JavaScript Object Notation

- JSON fornece um formato intermediário de dados que representa as informações específicos de um objeto qualquer em um formato fácil de ler e entender!

→ Dados de um objeto em JSON!

```
{  
  "id": 99,  
  "amount": 123.45  
}
```

- Usaremos JSON como o intermediário neste curso!

OBS: Existem outros formatos populares
como YAML e XML, mas o JSON é
mais rápido e fácil de usar e está na
maioria dos programas modernos!

• Testes:

TDD: Primeiro criamos os
testes e depois programamos!

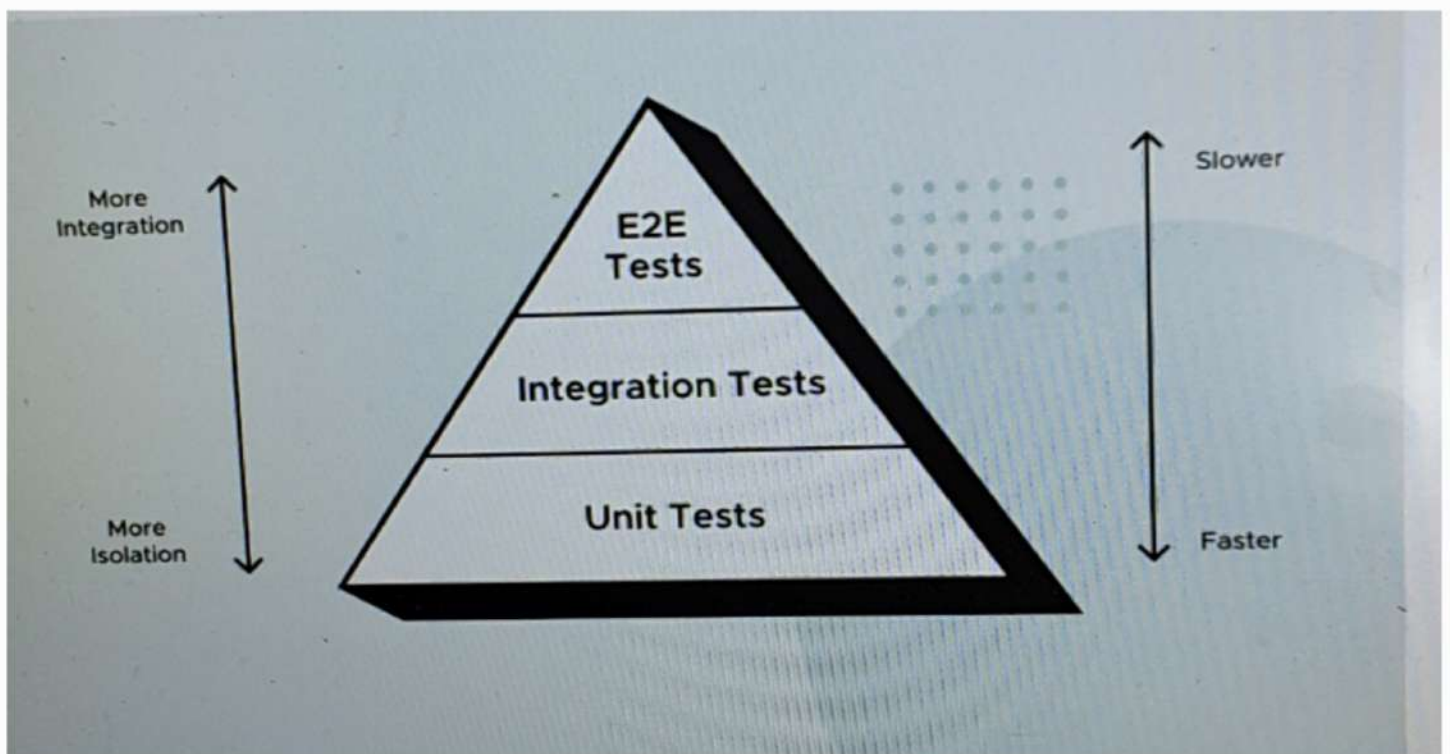
- Porque usar TDD?

- "want it to do"

- Quando definirmos o comportamento
esperado antes de implementar os
funcionalidades, conseguimos prazeres
o sistema com base no que queremos
que ele faça, ao invés do que já faz.

- Outro benefício da aplicação
per "Test-driving" é que
podemos escrever o mínimo de
código necessário para validar
a aplicação!

• A Pirâmide de Testes!



↳ Diferentes tipos de testes são feitos
em diferentes níveis de sistema!

- Cada minuto de teste há um
balança entre velocidade e
exatidão!

- A pirâmide de testes traz
a ideia entre custo e a confiança
de cada teste de modo hierárquico!

• Testes unitários:

- Um teste unitário executa
em uma "unidade" pequena
do sistema isolada do restante!

Devem ser simples e rápidos.

É importante ter uma alta proporção
de Testes Unitários na sua pirâmide

de testes, pois não é necessário para
projetar um software alternamente
com e sem testes de aceitação!

Unit Tests: A Unit Test exercises a small "unit" of the system that's isolated from the rest of the system. They should be simple and speedy. You want a high ratio of Unit Tests in your testing pyramid, as they're key to designing highly cohesive, loosely coupled software.

• Testes de integração:

Integration Tests: Integration Tests exercise a subset of the system and may exercise groups of units in one test. They are more complicated to write and maintain, and run slower than unit tests.

↳ Fazem parte de um "subconjunto" uma
parte maior do sistema que integram
grupos de unidades em um único teste.
São mais complexas e caras
para manter!

Minúsculas

End-to-End Tests: An End-to-End Test exercises the system using the same interface that a user would, such as a web browser. While extremely thorough, End-to-End Tests can be very slow and fragile because they use simulated user interactions in potentially complicated UIs. Implement the smallest number of these tests.

↳ Testes de "ponta a ponta" testam todo o
sistema!

Então testes agem como se fosse um usuário, usam até as mesmas interfaces não extremamente minuciosas e muito lentas e frágeis. Apesar de serem extremamente completos, devemos sempre implementar apenas o necessário dos testes!

The Red, Green, Refactor Loop

Software development teams love to move fast. So how do you go fast forever? By continuously improving and simplifying your code—refactoring. One of the only ways you can safely refactor is when you have a trustworthy test suite. Thus, the best time to refactor the code you're currently focusing on is during the TDD cycle. This is called the Red, Green, Refactor development loop:

↳ Uma forma para sempre programarmos rápida e com qualidade é continuarmos sempre simplificando e refatorando nosso código e o melhor momento para fazermos isso é durante o ciclo de TDD! Isso é chamado de Red, Green, Refactoring de refatoração!

1. **Red:** Write a failing test for the desired functionality.
2. **Green:** Implement the simplest thing that can work to make the test pass.
3. **Refactor:** Look for opportunities to simplify, reduce duplication, or otherwise improve the code without changing any behavior—to refactor.
4. Repeat!

De outra forma

Comportamento

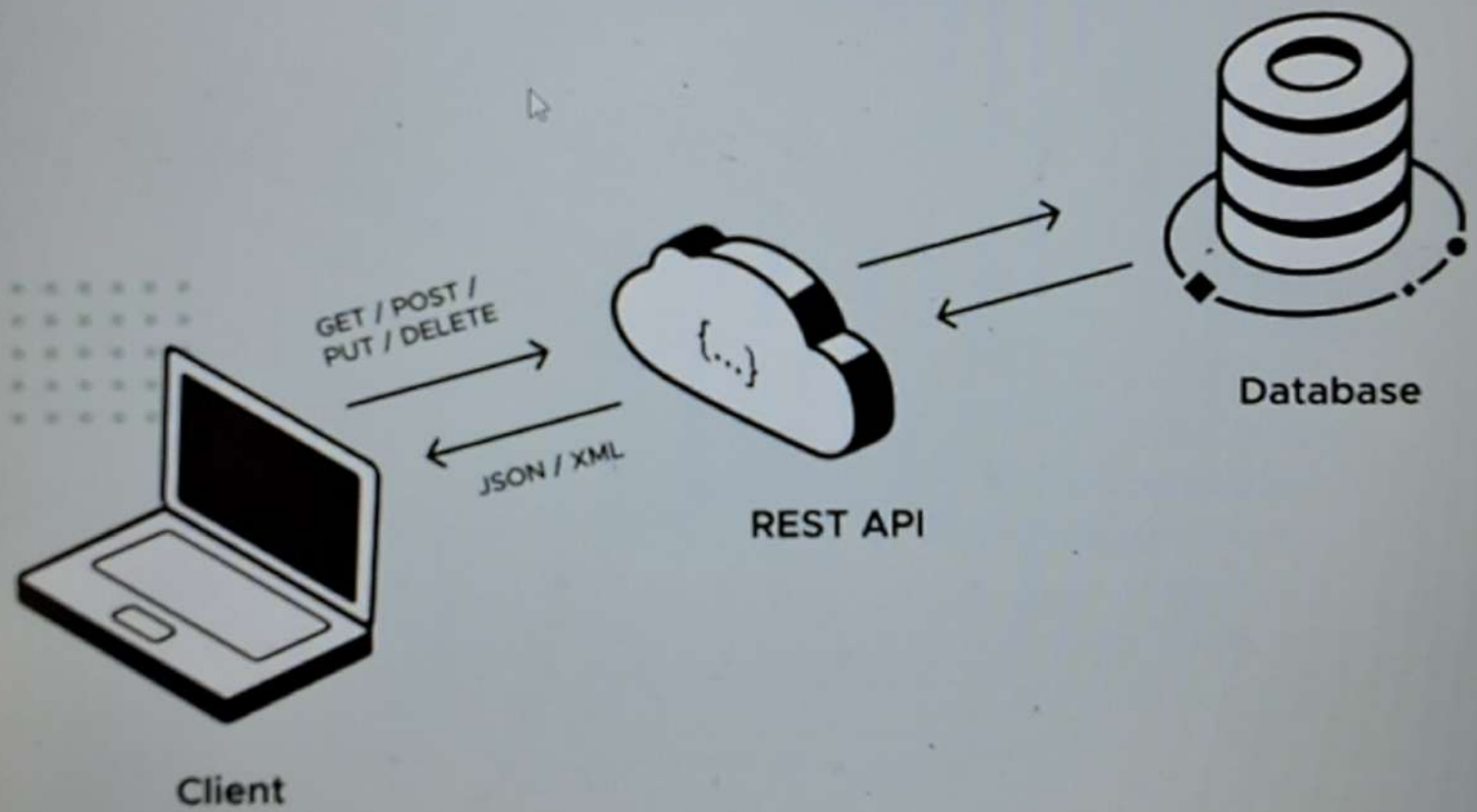
1- Vermelha: Escreva um teste falhando para a funcionalidade!

2- Verde: Implemente a solução mais simples possível que passe no teste!

3- Refatorar: Procure oportunidades para simplificar, reduzir a duplicação ou melhorar o código de outra maneira sem alterar o comportamento!

4- Repeat: Repita!

- Rest, cloud e HTTP:



Rest = Representational State Transfer

- O principal de uma API RESTful é gerenciar o estado de dados.

"Podemos pensar em "estado" como valores e "Resource Representation" como um objeto de dados."

- CRUD = Create, Read, Update e Delete

"Rest e mud são conceitos que andam juntos"

- Outro conceito muito associado a Rest é o HTTP.

HTTP: Protocolo de transferência de Hipertexto! Onde um solicitante envia uma requisição para um URL, onde um servidor recebe a requisição e a encaminha para um manipulador de requisições. O manipulador cria uma resposta, que é então enviada de volta para o solicitante!

- Os componentes de requisição e resposta:

- Requisições:

- Método (Também chamado de verbo)
- URL (Também chamado de endpoint)
- Corpo (Body)

- Resposta:

- Status do código
- Corpo (Body)

- Rest * Crud:

Para criar use HTTP: Post

Para ler use HTTP: Get

Para Atualizar use HTTP: PUT

Para Deletar use HTTP: Delete

The chart below has more details about RESTful CRUD operations.

CRUD

Operation	API Endpoint	HTTP Method	Response Status
Create	/cashcards	POST	201 (CREATED)
Read	/cashcards/{id}	GET	200 (OK)
Update	/cashcards/{id}	PUT	204 (NO CONTENT)
Delete	/cashcards/{id}	DELETE	204 (NO CONTENT)

URL

↳ Percebemos que "create" é o único que não precisa passar o "id" (index) pelo endpoint isto porque sempre iremos criar um novo e id será gerado ainda.

- Para "create" ou "Update" precisamos passar para a API o dado que será chamado de "corpo da requisição" ou "request body".

• Exemplo: Requisição GET por meio de uma end-point:

Let's use the example of a Read endpoint. For the Read operation, the URI (endpoint) path is /cashcards/{id}, where {id} is replaced by an actual Cash Card identifier, without the curly braces, and the HTTP method is GET.

In GET requests, the body is empty. So, the request to read the Cash Card with an id of 123 would be:

Request:

Method: GET

URL: http://cashcard.example.com/cashcards/123

Body: (empty)

COPY

The response to a successful Read request has a body containing the JSON representation of the requested Resource, with a Response Status Code of 200 (OK). Therefore, the response to the above Read request would look like this:

Response:

Status Code: 200

Body:

```
{
  "id": 123,
  "amount": 25.00
}
```

COPY

• Rest no Spring Boot:

- Spring's IoC container.
- Uma das principais funções do Spring é configurar e instanciar objetos. Estes objetos são chamados de "Beans do Spring" e geralmente são criados pelo Spring (ao invés de usar a palavra chave "new")

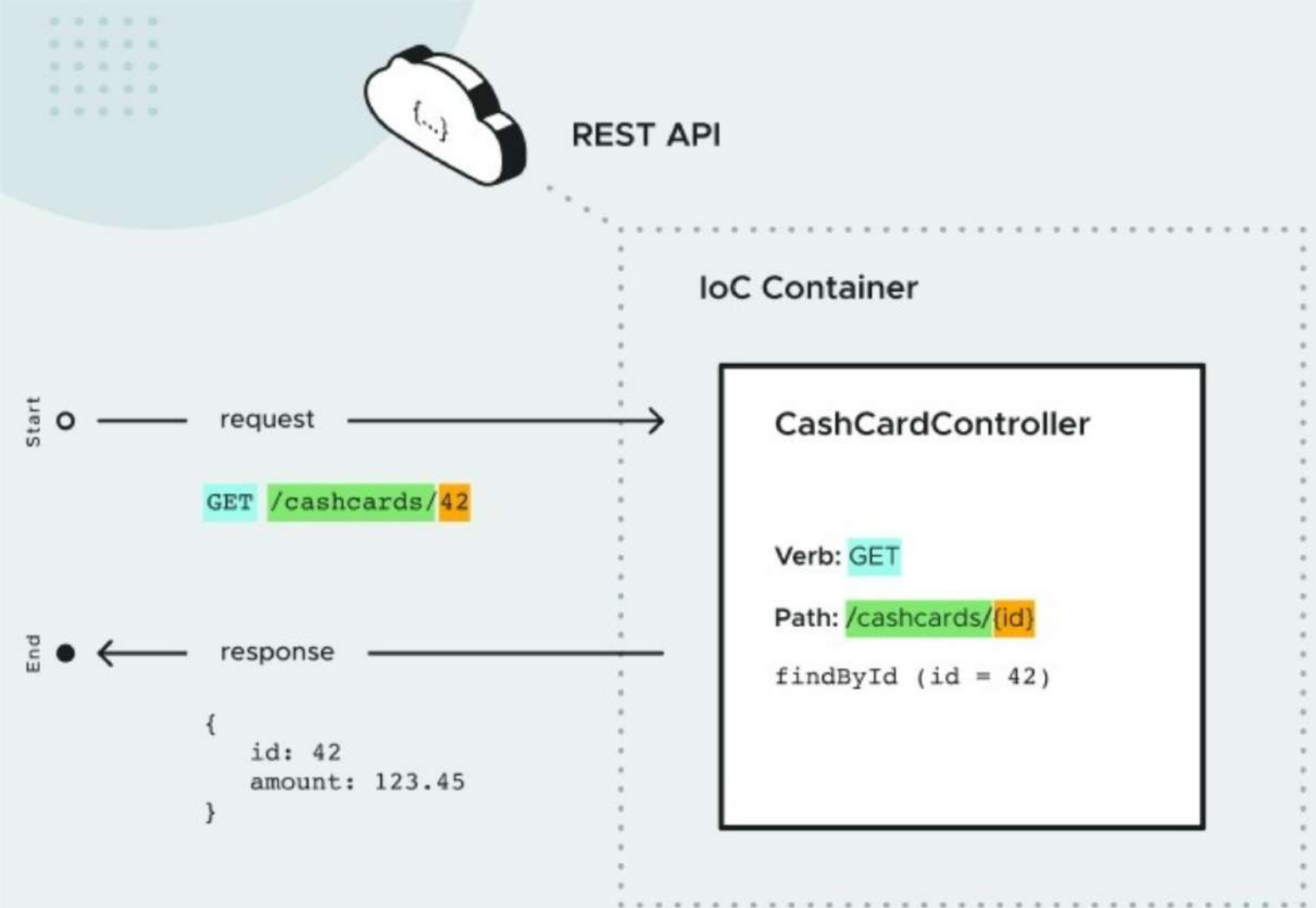
- Spring Web Controllers:
 - Em spring web rest são tratados por meio de "controllers"

Exemplo:

```
@RestController
```

```
class CashCardController {  
}
```

- "create a REST Controller".
 - O controller é injetado no Spring Web! Com as rotas API requests sendo tratados pelo controllers.



```
@RestController
class CashCardController {
    @GetMapping("/cashcards/{requestedId}")
    private ResponseEntity<CashCard> findById(@PathVariable Long requestedId) {
        CashCard cashCard = /* Here would be the code to retrieve the CashCard */;
        return ResponseEntity.ok(cashCard);
    }
}
```

@GetMapping → Dispatcher as Spring endpoint (URL PATH).

@PathVariable → Dispatcher as Spring and
it does resolve & passes! { requestedId } = requestedId

ResponseEntity OK -> Classe do Spring
com diversos respostas!

● Post:

- Introdução: Post não é exatamente um
padrão! É simplesmente uma maneira
de usar HTTP para realizar operações com dados!

- Idempotência (idempotência):

- Se executada mais de uma vez
resulta no mesmo resultado!
(No caso Post, resultaria os mesmos dados)

- Método POST permite um corpo, então
usaremos este corpo para enviar uma representação
JSON de um objeto.

• Exemplo de Post:

Request:

- Method: POST
- URI: /cashcards/
- Body:

```
{  
  amount: 123.45  
}
```

COPY

- Resposta em caso de criação bem-sucedida:
- Resposta em: 201 CREATED
- Uma resposta contém um "Status Code" e um "Header".
- Headers (cabeçalhos): Tem um nome e um valor! O padrão HTTP para Status Code 201 diz que o cabeçalho deve conter a URL do recurso criado!

• Exemplo de resposta:

Response:

- Status Code: 201 CREATED
- Header: Location=/cashcards/42

- Métodos Spring:

- O Spring nos fornece diferentes métodos recomendados para HTTP e REST.

- Um exemplo:

- `ResponseEntity.created()`

- Garante que o URL seja bem formado.
- Adiciona o cabeçalho "Location".
- Define o código de status automaticamente!

• Retornando uma lista com get!

- Muitas vezes precisamos obter vários objetos de uma única vez, então para isso é bem realizarmos um `unice request`!

- Para isso iremos usar um `array JSON`!

- Exemplo: ☐ - Indicando o array!

```
{
  "id": 1,
  "amount": 123.45
},
{
  "id": 2,
  "amount": 50.0
}
]
```


- Paginação e Sanitização:
 - Idealmente uma API deve retornar um tamanho limitado de dados, pois retornar de maneira ilimitada pode sobrecarregar a memória do cliente e do servidor.
 - Spring Data é paginativo e funcional?
 - O spring já oferece esta funcionalidade de paginação para limitar o tamanho de uma página!
 - Uma página tem um tamanho e um index!
 - Index 0, tamanho: 5
 $0 = \square 0 \Delta \square \star$
↳ O index 0 contém 5 "objetos"
 - $7 = \Delta \Delta \square \star X$
↳ O index 7 contém outros 5 "objetos"
 - O index das páginas sempre começará com 0!

- Nós podemos ordenar os dados dentro das páginas!

Exemplo:

Page 1:

1. \$1,000.00

2. \$50.00

3. \$20.00

→ Ordenando duas páginas, index 0 e 1 de tamanho 3!

Percebemos que
→ ordenamos as páginas
em ordem decrescente de valor!

Page 2:

1. \$10.00

2. \$0.19

• Exemplo de código para paginação em ordem decrescente:

```
Page<CashCard> page2 = cashCardRepository.findAll(
    PageRequest.of(
        1, // page index for the second page - indexing starts at 0
        10, // page size (the last page might have fewer items)
        Sort.by(new Sort.Order(Sort.Direction.DISC, "amount"))));
```

↳ Spring realizando a ordenação!

@GetMapping

```
private ResponseEntity<List<CashCard>> findAll(Pageable pageable) {
    Page<CashCard> page = cashCardRepository.findAll(
        PageRequest.of(
            pageable.getPageNumber(),
            pageable.getPageSize(),
            pageable.getSortOr(Sort.by(Sort.Direction.DISC, "amount"))));
    return ResponseEntity.ok(page.getContent());
}
```

• Implementação completa do método.

• Pageable: Recebe os parâmetros para consulta "page" e "size".
Se não fornecermos os parâmetros o Spring fornecerá o padrão:
page = 0, size = 20.

• getSortOr: Padrão de ordenação, mesmo sem um parâmetro para busca teremos sempre a mesma ordenação dos dados!

• page.getContent(): Retorna os valores contidos no objeto Page, para o chamador! Em nosso exemplo os CashCards.

• O que o object page contém?

```
{
  "content": [
    {
      "id": 1,
      "amount": 10.0
    },
    {
      "id": 2,
      "amount": 0.19
    }
  ],
  "pageable": {
    "sort": {
      "empty": false,
      "sorted": true,
      "unsorted": false
    },
    "offset": 3,
    "pageNumber": 1,
    "pageSize": 3,
    "paged": true,
    "unpaged": false
  },
  "last": true,
  "totalElements": 5,
  "totalPages": 2,
  "first": false,
  "size": 3,
  "number": 1,
  "sort": {
    "empty": false,
    "sorted": true,
    "unsorted": false
  },
  "numberOfElements": 2,
  "empty": false
}
```

• Object Page em formato JSON:

• Content é onde está o array com o novo conteúdo!

• Estes demais campos contém informações sobre como esta página está relacionada a outras páginas na consulta!

• Devemos definir um "contrato" para enviarmos apenas as informações desejadas pelo usuário, sem tudo o resto que nem nas paginações!

• Serializar:

- "Serialization" é um conceito importante, onde transformamos um valor em outro formato para podermos transportá-lo, ou armazená-lo.

• Um pouco sobre segurança web:

- Spring Security: Oferece diversos recursos para melhorar a segurança de aplicações!

- Um usuário de uma APT pode ser uma pessoa ou até mesmo um sistema que irá conectar a nossa APT!

- Uma das formas mais básicas de se proteger um sistema é integrando alguma forma de autenticação! (Ex: login e senha)

- HTTP é um protocolo sem estado, e não é fácil senti-lo autenticar o "Principal".
Toda vez que necessitamos realizar alguma ação! Por isso temos "Sessions".
União com outro sistema

- Uma sessão serve para manter o estado do Principal entre várias operações que precisamos realizar.
(Uma sessão pode ser implementada de várias maneiras)

- Diversas tentativas maliciosas podem acontecer durante este período da sessão!
Para proteger o sistema temos que adotar diversas práticas, um muito utilizado em aplicações web é o "Filter Chain" (Cadeia de filtros).

- Filter Chain: Uma requisição é processada em etapas (Filtros) antes de ir para o controlador / Servlet

- O Spring Security implementa a autenticação na cadeia de filtros, insere um filtro que verifica a autenticação do principal e retorna 401 (Não autorizado) caso não esteja autenticado!

- Autenticação $\neq$ Autorização:

A autenticação ocorre após o principal ser autenticado, isto porque é muito comum em um sistema que determinados funções não possam ser exercidas por um principal em específico!

- O String Security também oferece recursos para lidar com autorização.

↳ Controle de Acesso Baseado em Funções.
(RBAC)

- RBAC: Uma entidade "Principal" possui diversos funções (Roles). A entidade não terá acesso a estas funções de acordo com seu nível de autorização!

- Same Origin Policy (SOP): Política de proteção determina que apenas scripts contidos em uma página web podem fazer requisições para a origem (URL) da página.

(A Política SOP é crucial para segurança de sites, pois, sem ela qualquer pessoa poderia criar uma página web entendendo um script que fornece requisições para qualquer endereço!)
↳ SOP é uma política colocada pelo navegador!

OBS: Às vezes temos um sistema que consiste em vários executados em vários máquinas com URLs diferentes! O CORS, compartilhamento de recursos de origem cruzada, é uma maneira que os navegadores usam para flexibilizar a SOP!

↳ Spring oferece a anotação @CrossOrigin!
(precisamos configurá-la com cuidado)

- Alguns exemplos de ataques comuns:

- CSRF: ^{no cross-site} Session Riding (Roubo de sessão), é possibilitado pelos cookies, onde quem tem um código malicioso envia uma requisição a um servidor onde o usuário já está autenticado.

↳ Para nos protegermos de CSRF, podemos usar um token CSRF, é um token exclusivo gerado a cada requisição, o que dificulta a invasão de um agente externo na "caverna"!

↳ Spring Security possui suporte integrado para tokens CSRF, que já vem habilitado por padrão!

- Cross-Site Scripting (XSS):

De alguma forma explorando alguma vulnerabilidade do sistema para executar um código arbitrário (existem diversas maneiras de fazer isso) Um exemplo é salvar uma string em um banco de dados. XSS não depende de autenticação e pode causar grandes estragos, pois podem também ser executados nos servidores e não apenas no usuário!

↳ Spring também oferece recursos para proteger contra isso.

● Implementando PUT:

- Put and Patch,

- Ambos servem para update, mas de maneiras diferentes.

- Put: Atualiza todo registro.

- Patch: Atualização parcial do registro.

(Normalmente Patch é usado para registros grandes, para não sobrecarregar o servidor nem necessidade)

● Tabela para ajudar a escolher qual utilizar:

HTTP Method	Operation	Definition of Resource URI	What does it do?	Response Status Code	Response Body
POST	Create	Server generates and returns the URI	Creates a sub-resource ("under" or "within" the passed URI)	201 CREATED	The created resource
PUT	Create	Client supplies the URI	Creates a resource (at the Request URI)	201 CREATED	The created resource
PUT	Update	Client supplies the URI	Replaces the resource: The entire record is replaced by the object in the Request	204 NO CONTENT	(empty)
PATCH	Update	Client supplies the URI	Partial Update: modify only fields included in the request on the existing record	200 OK	The updated resource

↳ Dependendo do tipo de nossa API, podemos via-la de maneiras muito diferentes!

• Implementing DELETE:

→ Nome relitório.

Request:

- Verb: DELETE
- URI: /cashcards/{id}
- Body: (empty)

• Retornar esperados:

Response Code	Use Case
204 NO CONTENT	• The record exists, and • The Principal is authorized, and • The record was successfully deleted.
404 NOT FOUND	• The record does not exist (a non-existent ID was sent).
404 NOT FOUND	• The record does exist but the Principal is not the owner.

↳ "404" para ambos, pois não podemos "avisar" a um proximiário que ele está no caminho errado!

- Delete Hard e Delete Soft?

- Hard: Deleta tudo completamente do banco de dados de maneira direta!

- Soft: Faz um delete "lógico" onde o banco de dados ainda armazena, mas é marcado como "deletado", desta maneira podemos implementar uma regra caso haja entendimento, ou podemos usar como mecanismos de segurança!

- Registro de auditoria e Campos de auditoria:

- Muitas da vez devemos também armazenar alguns dados referentes as operações de delete! Como data ou quem deletou e etc.

- Campos de auditoria: Adiciona erros "após" no próprio campo de delete.

- Registro de auditoria: Fica como um log, armazenamos mais "genéricas": como um log com todos os erros, ou operações!

- Maven / Gradle Groovy / Gradle Kotlin.

- Ambos os 3 são ferramentas de gerenciamento de projetos e dependências Java!

- Eles baixam as bibliotecas.

- Eles organizam o projeto.

- Eles compilam e rodam o projeto.

Lembrete: Ambos é necessário baixar e adicionar as variáveis do ambiente!

• Entendendo o código:

```
package com.example.restservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RestServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(RestServiceApplication.class, args);
    }
}
```

Package em que a classe pertence!

Importa as classes do Spring.

"Annotation":

É durante a execução dig ao Spring como aquilo deve funcionar!

Adiciona a classe

Método para iniciar o aplicativo.

Classe.

@SpringBootApplication é uma anotação que adiciona outras anotações: @Configuration, @EnableAutoConfiguration e @ComponentScan.

```
package com.example.restservice;

public record Greeting(long id, String content) { }
```

Empacotamento

Dados que vai enviar!

Nome da classe!

• Diz que é uma classe **IMUTÁVEL** de dados! Uma vez atribuídos pelo construtor os dados permanecem! (via automaticamente os getters, equals(), hashCode(), toString())!
Não aceita setters!
Podemos criar métodos!

